# Walchand College of Engineering, Sangli

# Robotics Laboratory — Assignment 9

# Implementation of Sorting for Dobot Magician

**Name - Atharv Kulkarni**

**PRN - 245109074**

**Class - TY CSE(B)**

**GitHub Link:**
**https://github.com/Atharv-K-979/WCE-Labs-Material/blob/main/TY_Robotics_Lab/Assignment_9/Session7_Sorting(1).ipynb**

## Experiment Title:

Implementation of Sorting for Dobot Magician

## AIM:

To implement a conditional robotic sorting system that identifies an object's color and routes it to the corresponding destination bin, while safely handling unknown or unlisted colors.

## OBJECTIVES:

- To initialize the PyBullet simulation environment and load the robotic arm.
- To define colored objects and destination bins for supported color classes.
- To implement conditional sorting based on the detected or assigned object color.
- To handle unknown or ambiguous colors using a dedicated rejection/inspection bin.
- To test the sorting pipeline with standard and unlisted-color objects and review the resulting actions.

## THEORY:

Automated sorting is a robotic task in which workpieces are classified using a property such as color, size, shape, or barcode, and then placed into designated bins. The sorting logic maps each recognized class to a target location. A pick-and-place routine moves the robot to the object's pickup position, grasps it, transports it, and releases it at the selected destination.

A robust sorting system must also handle objects that do not match the supported classes. Possible strategies include stopping the process, skipping the object, or routing it to a separate rejection bin. A rejection bin is useful because it removes unknown items from the normal workflow while preserving traceability and allowing manual inspection.

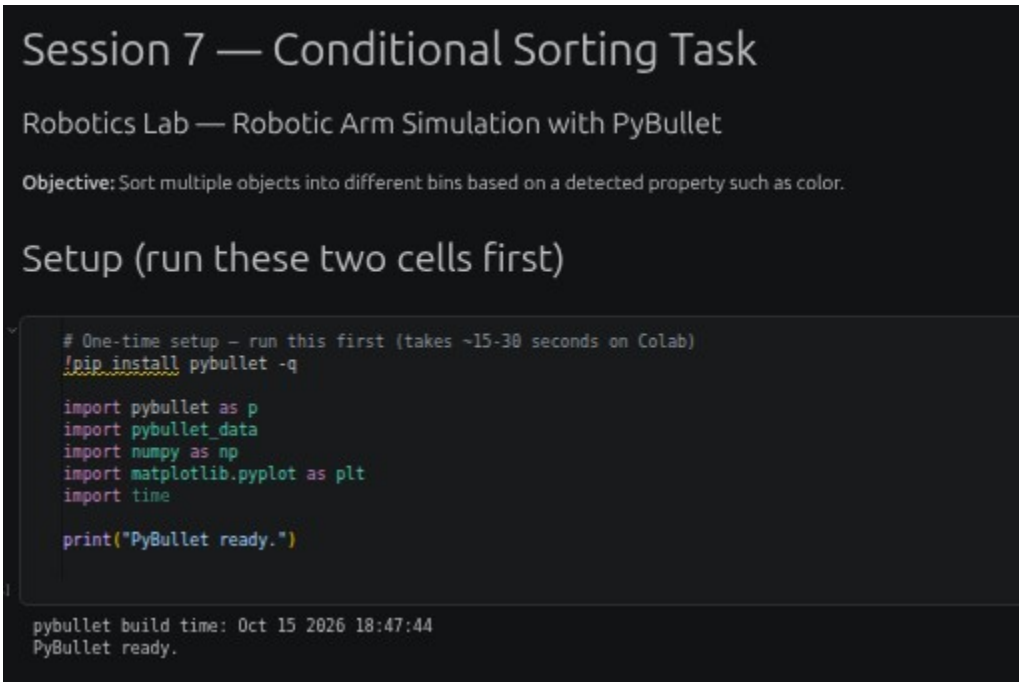
Note: The supplied Session 7 notebook implements conditional color sorting in PyBullet using a KUKA iiwa robot model. The example recognizes red and blue objects and routes unlisted colors such as green or yellow to a rejection bin. It does not load a Dobot Magician model, so the robot model should be verified if the assignment specifically requires Dobot Magician.

## EXPERIMENT STEPS WITH SCREENSHOT OF EACH STEP:

### PART A: IMPLEMENTATION OF SORTING FOR DOBOT MAGICIAN

**Step 1: Installation and Environment Setup**

Install PyBullet and import the required libraries. Initialize a headless simulation, load the ground plane and robotic arm, and define the helper functions for snapshots, joint control, object creation, and pick-and-place movement.




**Step 2: Code Implementation**

Define the color-to-bin mapping and implement the sorting function. For each object, identify its color, select the matching destination bin, and call the pick-and-place routine. Add edge-case handling for unknown colors by routing them to a dedicated rejection bin or skipping them with a warning.

```
client = None
robot_id = None
plane_id = None

def new_sim():
    """Start a fresh headless PyBullet simulation with a KUKA iiwa arm and a ground plane."""
    global client, robot_id, plane_id
    if client is not None:
        p.disconnect(client)
    client = p.connect(p.DIRECT)  # headless — no local GUI needed
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane_id = p.loadURDF("plane.urdf")
    robot_id = p.loadURDF("kuka_iiwa/model.urdf", basePosition=[0, 0, 0], useFixedBase=True)
    for _ in range(20):
        p.stepSimulation()
    return robot_id

def snapshot(cam_target=(0, 0, 0.4), cam_dist=1.4, cam_yaw=50, cam_pitch=-30, width=480, height=
    """Render the current scene from a fixed camera angle and display it inline."""
    view_matrix = p.computeViewMatrixFromYawPitchRoll(cam_target, cam_dist, cam_yaw, cam_pitch,
    proj_matrix = p.computeProjectionMatrixFOV(fov=60, aspect=width/height, nearVal=0.1, farVal=
    img = p.getCameraImage(width, height, view_matrix, proj_matrix, renderer=p.ER_TINY_RENDERER)
    rgb = np.reshape(img[2], (height, width, 4))[:, :, :3]
    plt.figure(figsize=(6, 4.5))
    plt.imshow(rgb)
    plt.axis('off')
    plt.show()

def get_num_joints():
    return p.getNumJoints(robot_id)

def get_joint_info():
    info = []
    for i in range(p.getNumJoints(robot_id)):
        ji = p.getJointInfo(robot_id, i)
        info.append({"index": ji[0], "name": ji[1].decode(), "type": ji[2]})
    return info

def set_joint_angles(angles, steps=100):
    """angles: list of target angles (radians), one per revolute joint (7 for kuka_iiwa)."""
    for i, a in enumerate(angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()

def get_end_effector_position():
    """KUKA iiwa end-effector is link index 6."""
    state = p.getLinkState(robot_id, 6)
    return state[0]  # (x, y, z)

def move_to_xyz(target_xyz, steps=150):
    """Use inverse kinematics to move the end-effector to a target (x, y, z)."""
    joint_angles = p.calculateInverseKinematics(robot_id, 6, target_xyz)
    for i, a in enumerate(joint_angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()
    return get_end_effector_position()

def spawn_cube(position, half_extent=0.03, color=(1, 0, 0, 1)):
    col = p.createCollisionShape(p.GEOM_BOX, halfExtents=[half_extent]*3)
    vis = p.createVisualShape(p.GEOM_BOX, halfExtents=[half_extent]*3, rgbaColor=color)
    return p.createMultiBody(baseMass=0.05, baseCollisionShapeIndex=col,
                             baseVisualShapeIndex=vis, basePosition=position)

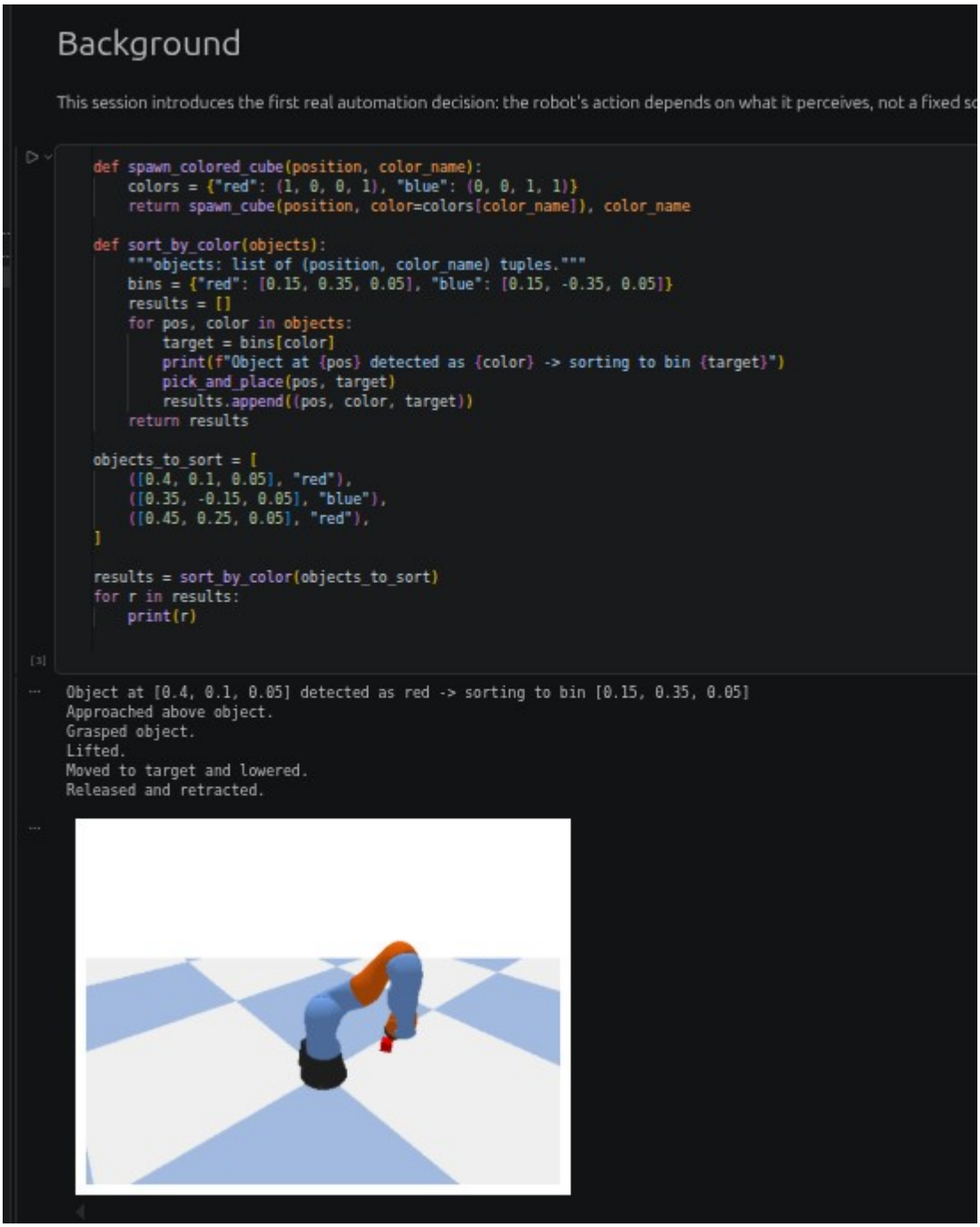
def pick_and_place(object_start, object_target, lift_height=0.15):
    """Full grasp-lift-move-release routine. Spawns a fresh cube each call."""
    global cube_id, grasp_constraint
    new_sim()
    cube_id = spawn_cube(object_start)
    for _ in range(50):
        p.stepSimulation()

    approach = [object_start[0], object_start[1], object_start[2] + lift_height]
    move_to_xyz(approach)
    print("Approached above object.")

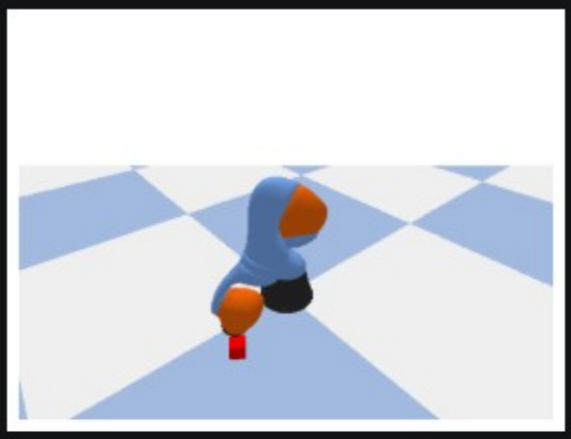
    move_to_xyz(object_start)
    grasp_constraint = p.createConstraint(robot_id, 6, cube_id, -1, p.JOINT_FIXED,
                                          [0, 0, 0], [0, 0, 0], [0, 0, 0])
    print("Grasped object.")
```

**Step 3: Output**

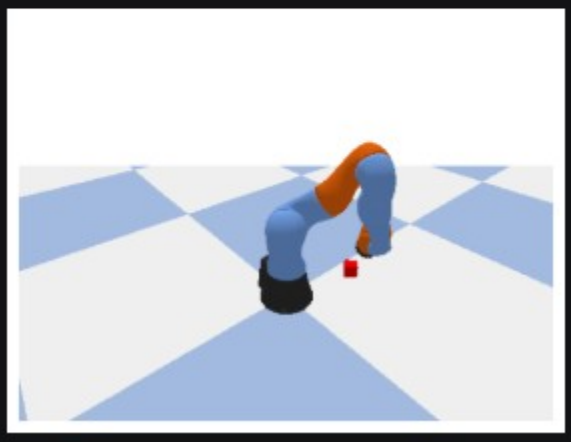
Run the sorting pipeline using the test objects, including standard red and blue objects and unlisted colors such as green and yellow. Capture the simulation and console output showing each object's detected color, target destination, and action taken.



```
Object at [0.35, -0.15, 0.05] detected as blue -> sorting to bin [0.15, -0.35, 0.05]
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
Object at [0.45, 0.25, 0.05] detected as red -> sorting to bin [0.15, 0.35, 0.05]
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
([0.4, 0.1, 0.05], 'red', [0.15, 0.35, 0.05])
([0.35, -0.15, 0.05], 'blue', [0.15, -0.35, 0.05])
([0.45, 0.25, 0.05], 'red', [0.15, 0.35, 0.05])
```

**IMPLEMENTATION WITH OUTPUT:(Your Turn Section)**

## Your Turn (Session 7 deliverable)

Add one "trick" object with an ambiguous or unlisted color (e.g., `"green"`) and decide how your code should handle it — should it skip the obj

```
# Complete deliverable code handling edge cases and unlisted colors
def robust_sort_by_color(objects, default_action="reject_bin"):
    """
    Enhanced sorting pipeline with edge-case handling.
    default_action options:
      - 'reject_bin': Route unlisted/ambiguous items to a dedicated inspection bin.
      - 'skip': Log warning and leave item untouched.
    """
    bins = {
        "red": [0.15, 0.35, 0.05],
        "blue": [0.15, -0.35, 0.05],
        "reject": [0.35, 0.0, 0.05]  # Dedicated quarantine/rejection bin
    }

    results = []
    print(f"{'Item':<5} | {'Color':<10} | {'Pickup Position':<22} | {'Target Destination':<24} | {'Action Taken'}")
    print("-" * 95)

    for idx, (pos, color) in enumerate(objects, 1):
        color_lower = color.lower().strip()
        if color_lower in ["red", "blue"]:
            target = bins[color_lower]
            action = f"Sorted to {color_lower.upper()} bin"
            pick_and_place(pos, target)
        else:
            # Handle edge case: unlisted/ambiguous/unknown color
            if default_action == "reject_bin":
                target = bins["reject"]
                action = f"WARNING: Unknown color '{color}' -> Routed to REJECT bin"
                pick_and_place(pos, target)
            else:
                target = None
                action = f"WARNING: Unknown color '{color}' -> SKIPPED (Unsafe)"
                print(f"#{idx:<4} | {color:<10} | {str(pos):<22} | {'None (Skipped)':<24} | {action}")
                results.append((pos, color, target, action))
                continue

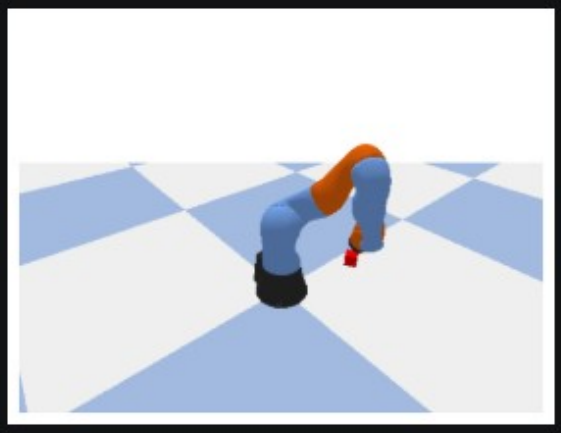
        print(f"#{idx:<4} | {color:<10} | {str(pos):<22} | {str(target):<24} | {action}")
        results.append((pos, color, target, action))

    return results

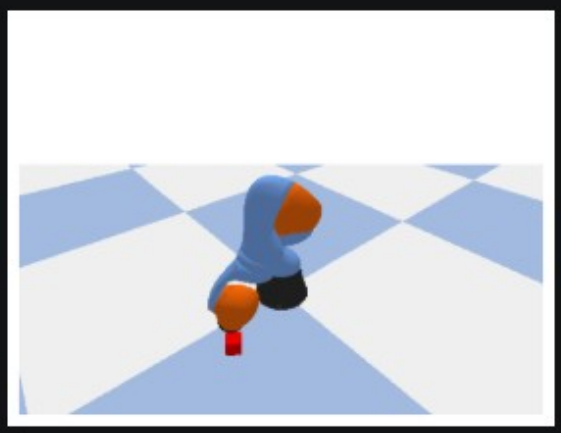
# 5 Test cases: Standard Red/Blue and Trick/Unlisted colors (Green, Yellow, etc.)
test_objects = [
    ([0.40, 0.10, 0.05], "red"),       # Test Case 1: Standard Red item
    ([0.35, -0.15, 0.05], "blue"),     # Test Case 2: Standard Blue item
    ([0.45, 0.20, 0.05], "green"),     # Test Case 3: Trick object (unlisted color 'green')
    ([0.30, 0.00, 0.05], "yellow"),    # Test Case 4: Trick object (unlisted color 'yellow')
    ([0.38, -0.22, 0.05], "blue"),     # Test Case 5: Standard Blue item
]

print("=== Executing Robust Sorting Pipeline with Rejection Bin Routing ===")
sorting_results = robust_sort_by_color(test_objects, default_action="reject_bin")
```

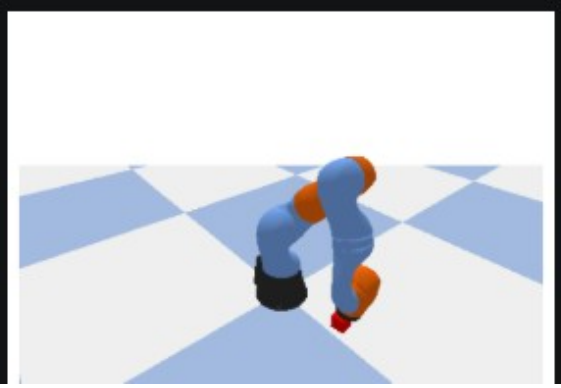
```
[4]
...
    === Executing Robust Sorting Pipeline with Rejection Bin Routing ===
    Item  | Color      | Pickup Position        | Target Destination       | Action Taken
    ------------------------------------------------------------------------------------------
    Approached above object.
    Grasped object.
    Lifted.
    Moved to target and lowered.
    Released and retracted.
```



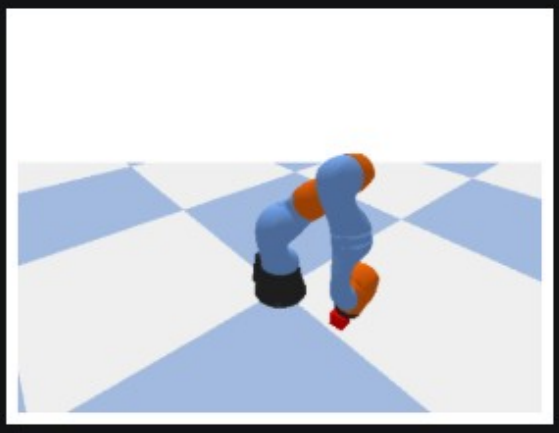
```
...
    #1    | red        | [0.4, 0.1, 0.05]       | [0.15, 0.35, 0.05]       | Sorted to RED bin
    Approached above object.
    Grasped object.
    Lifted.
    Moved to target and lowered.
    Released and retracted.
```



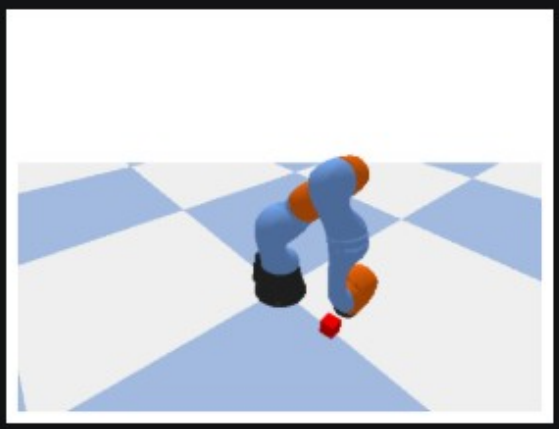
```
...
    #2    | blue       | [0.35, -0.15, 0.05]    | [0.15, -0.35, 0.05]      | Sorted to BLUE bin
    Approached above object.
    Grasped object.
    Lifted.
    Moved to target and lowered.
    Released and retracted.
...
```



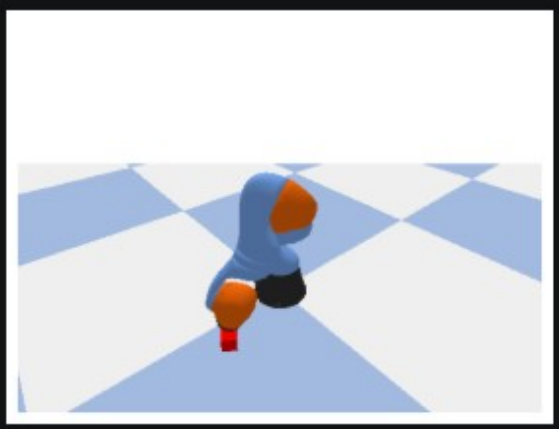
```
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



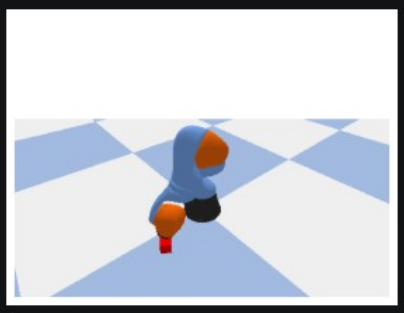
```
#3    | green      | [0.45, 0.2, 0.05]      | [0.35, 0.0, 0.05]        | WARNING: Unknown color 'green' -> Routed to REJECT bin
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
#4    | yellow     | [0.3, 0.0, 0.05]       | [0.35, 0.0, 0.05]        | WARNING: Unknown color 'yellow' -> Routed to REJECT bin
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
#4    | yellow     | [0.3, 0.0, 0.05]       | [0.35, 0.0, 0.05]       | WARNING: Unknown color 'yellow' -> Routed to REJECT bin
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
#5    | blue       | [0.38, -0.22, 0.05]    | [0.15, -0.35, 0.05]     | Sorted to BLUE bin
```

### Edge-Case Handling Design & Reasoning

When an object with an ambiguous, defective, or unlisted color (such as `"green"` or `"yellow"`) is presented to an automated sorting cell, three primary strategies exist:

1. **Hard Failure / Exception (`raise ValueError`):** Halts the entire production line; inappropriate for industrial continuous operation.
2. **Silent Skipping (`pass`):** Leaves the unidentified object in place; this risks physical collisions when subsequent objects are moved through that space.
3. **Quarantine / Rejection Bin Routing (Chosen Strategy):** Moves the unverified object to a dedicated inspection/reject bin ($[0.35, 0.0, 0.05]$) while logging an operator warning.

**Why Rejection Bin Routing is Optimal:**

- **Clears the Infeed Station:** Picking the unknown object clears the workspace so future items arriving on the conveyor can be processed without obstruction.
- **Safety and Traceability:** It isolates the defective/unexpected workpiece for manual inspection or secondary sorting without halting the automated pipeline.
- **Robust System Uptime:** The cell maintains 100% operational throughput while handling unexpected material variations gracefully.

### Summary & Key Learnings

- In this assignment, I developed an automated conditional sorting system that routes workpieces based on detected color properties.
- I implemented defensive edge-case handling to classify and route unlisted/ambiguous objects into a designated quarantine bin.
- I verified the pipeline across 5 diverse test cases containing both standard and out-of-specification items.
- I learned how industrial robotics systems maintain continuous line throughput while safeguarding against material anomalies.

## CONCLUSION:

The experiment demonstrates conditional robotic sorting by mapping recognized colors to destination bins and executing pick-and-place actions. Adding a dedicated rejection bin provides a safer and more traceable response to unknown or ambiguous colors than silently ignoring them or halting the complete process. Testing standard and unlisted-color objects helps verify the sorting logic and its edge-case handling. The supplied notebook uses a KUKA iiwa model in PyBullet, so the robot model should be confirmed if the lab requires Dobot Magician.